BST Student Record System

Name : *Jalkiran Kaur* | ***Roll No :*** *2556582* | ***Branch :*** *B.Tech(AI&DS)*
DSA Mini Project (PPF Week 9)

BST Structure

Structure: Each node stores a student record (ID, name, department) and two pointers, left and right. BST rule: left subtree keys < node key < right subtree keys.

Algorithms

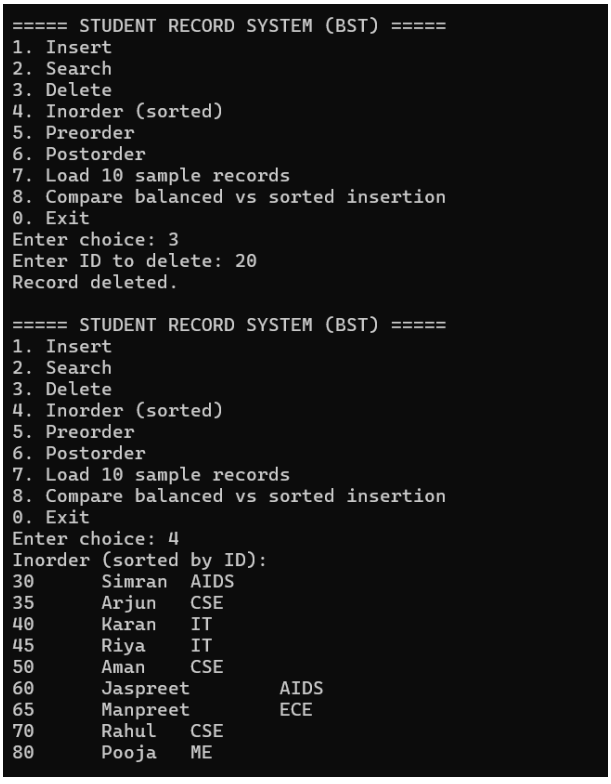
1. **Insert:** start at the root, go left if the key is smaller and right if larger, and place the node at the first NULL. Equal key means reject.
2. **Search:** same path as insert, stopping when the key is found or NULL is reached.
3. **Delete:**
    - Leaf: remove the node.
    - One child: replace the node with its child.
    - Two children: copy the inorder successor (smallest in right subtree) into the node, then delete the successor.
4. **Traversals:** Inorder = Left, Root, Right (gives sorted order). Preorder = Root, Left, Right. Postorder = Left, Right, Root.

Test Cases (with sample data)

No.	Test	Action	Expected Result
1	Delete leaf	Delete 20	Node removed, parent's left becomes NULL
2	Delete one child	Delete 60	Child 65 takes its place
3	Delete two children	Delete 30	Replaced by inorder successor 35
4	Duplicate insert	Insert 50	"Duplicate rejected"
5	Failed search	Search 99	"ID 99 not found"

Test Case Screenshots

1. Delete leaf (20)



2. Delete one child (60)

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 3
Enter ID to delete: 60
Record deleted.

===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 4
Inorder (sorted by ID):
30      Simran  AIDS
35      Arjun   CSE
40      Karan   IT
45      Riya    IT
50      Aman    CSE
65      Manpreet        ECE
70      Rahul   CSE
80      Pooja   ME
```

3. Delete two children (30)

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 3
Enter ID to delete: 30
Record deleted.

===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 4
Inorder (sorted by ID):
35      Arjun   CSE
40      Karan   IT
45      Riya    IT
50      Aman    CSE
65      Manpreet        ECE
70      Rahul   CSE
80      Pooja   ME
```

4. Duplicate insert (50)

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 1
Enter ID: 50
Enter name: Jalkiran
Enter department: AIDS
Error: ID 50 already exists. Duplicate rejected.
```

5. Failed search (99)

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 2
Enter ID to search: 99
ID 99 not found.
```

Program Output

- Insertion and inorder

```
C:\Users\Acer\OneDrive\Desk

===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 7
10 sample records inserted.

===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 4
Inorder (sorted by ID):
20      Neha    ECE
30      Simran  AIDS
35      Arjun   CSE
40      Karan   IT
45      Riya    IT
50      Aman    CSE
60      Jaspreet        AIDS
65      Manpreet        ECE
70      Rahul   CSE
80      Pooja   ME
```

- Successful search

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 2
Enter ID to search: 40
Record found:
40      Karan   IT
```

- Preorder

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 5
Preorder:
50      Aman    CSE
30      Simran  AIDS
20      Neha    ECE
40      Karan   IT
35      Arjun   CSE
45      Riya    IT
70      Rahul   CSE
60      Jaspreet        AIDS
65      Manpreet        ECE
80      Pooja   ME
```

- Postorder

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 6
Postorder:
20      Neha    ECE
35      Arjun   CSE
45      Riya    IT
40      Karan   IT
30      Simran  AIDS
65      Manpreet        ECE
60      Jaspreet        AIDS
80      Pooja   ME
70      Rahul   CSE
50      Aman    CSE
```

Insertion Order Comparison

Insertion order	Comparisons	Tree height
Balanced (4,2,6,1,3,5,7)	10	3
Sorted (1,2,3,4,5,6,7)	21	7

```
===== STUDENT RECORD SYSTEM (BST) =====
1. Insert
2. Search
3. Delete
4. Inorder (sorted)
5. Preorder
6. Postorder
7. Load 10 sample records
8. Compare balanced vs sorted insertion
0. Exit
Enter choice: 8
Balanced order (4,2,6,1,3,5,7) : 10 comparisons
Sorted order   (1,2,3,4,5,6,7) : 21 comparisons
```

Sorted insertion makes the tree skewed, so comparisons grow as n(n-1)/2 instead of about n log n

Complexity

Operation	Average	Worst
Insert / Search / Delete	O(log n)	O(n)
Traversal	O(n)	O(n)

Space: O(n) for storing nodes, plus O(h) for recursion.

BST Vs AVL

Why a BST becomes O(n): If IDs are inserted in sorted order (1, 2, 3, ...), every node goes to the right. The tree becomes a chain like a linked list, with height n, so each operation may visit n nodes.

How AVL fixes it: An AVL tree is a self-balancing BST. For every node, the height difference between left and right subtrees (balance factor) must be -1, 0 or +1. If an insert or delete breaks this, the tree performs rotations (LL, RR, LR, RL) to restore balance. Height stays O(log n), so every operation is O(log n) even in the worst case.